

Universidad Peruana de Ciencias Aplicadas



TRABAJO FINAL

PROGRAMACIÓN CONCURRENTE Y DISTRIBUIDA

TÍTULO:

Sistema paralelo para la predicción de alta demanda eléctrica doméstica

CARRERA:

Carrera de Ciencias de la Computación

Sección: 16661

Alumnos:	
Código	Nombres y apellidos
U201613422	Acuña Villegas, Omar Junior
U201925837	Chui Sanchez, Rafael Tomas
U202222148	Pariona Rojas, Axel Yamir

ÍNDICE

1. RESUMEN DEL TRABAJO.....	3
2. DESCRIPCIÓN DEL PROBLEMA Y MOTIVACIÓN.....	3
2.1 Contexto del problema.....	3
2.2 Problema identificado.....	4
2.3 Motivación.....	4
2.4 Relación con sostenibilidad.....	4
3. OBJETIVOS.....	4
3.1 Objetivo general.....	5
3.2 Objetivos específicos.....	5
4. DESCRIPCIÓN DEL DATASET.....	5
4.1 Fuente del dataset.....	5
4.2 Descripción general.....	6
4.3 Variables del dataset.....	6
4.4 Justificación de elección del dataset.....	7
5. LIMPIEZA Y ANÁLISIS DE DATOS.....	7
5.1 Problemas identificados en los datos.....	7
5.2 Estrategia de limpieza implementada.....	7
5.3 Resultados de limpieza.....	8
5.4 Variables derivadas.....	9
5.5 Fórmula de consumo no medido.....	9
5.6 Estadísticas descriptivas principales.....	10
5.7 Evidencias gráficas del análisis exploratorio.....	10
5.8 Horas con mayor consumo promedio y tasa de alta demanda.....	14
6. DISEÑO DEL MODELO DE MACHINE LEARNING.....	15
6.1 Tipo de problema.....	15
6.2 Variable objetivo.....	15
6.3 Variables predictoras.....	16
6.4 Modelo seleccionado.....	16
6.5 Métricas de evaluación.....	17
7. PARALELIZACIÓN DEL CÁLCULO.....	20
7.1 Justificación de paralelización.....	20
7.2 Etapas paralelizadas.....	20
7.3 Patrones de concurrencia utilizados.....	21
7.4 Métricas de rendimiento de la ejecución concurrente.....	22
7.5 Consideraciones sobre seguridad concurrente.....	23
8. ARQUITECTURA FUNCIONAL DEL SISTEMA.....	23
8.1 Componentes técnicos.....	24
8.2 Flujo de ejecución.....	25
8.3 Preparación para evolución futura del sistema.....	25
9. EVIDENCIAS DE IMPLEMENTACIÓN.....	25
9.1 Evidencia de ejecución final.....	26
9.2 Evidencias generadas por el sistema.....	26
9.3 Estructura de evidencias del proyecto.....	27

9.4 Evidencias del proceso de prueba y validación.....	28
9.5 Evidencias de benchmark y rendimiento.....	28
9.6 Comandos de reproducción.....	29
9.7 Evidencia de desarrollo iterativo.....	30
10. GESTIÓN DEL PROYECTO EN GITHUB.....	30
11. CONCLUSIONES.....	31
12. RECOMENDACIONES.....	32
13. BIBLIOGRAFÍA.....	33
14. ANEXOS.....	33
Anexo A. Link del repositorio en GitHub.....	33
Anexo B. Archivos de resultados principales.....	33
Anexo C. Extracto de salida de predicciones.....	34
Anexo D. Extracto de historial de entrenamiento.....	35
Anexo E. Evidencias de benchmark concurrente.....	35
Anexo F. Estructura de carpetas esperada para revisión.....	36
Anexo G. Evidencias visuales de ejecución y resultados.....	36

1. RESUMEN DEL TRABAJO

El presente trabajo desarrolla un sistema concurrente en Go para analizar y predecir momentos de alta demanda eléctrica doméstica a partir del dataset *Individual Household Electric Power Consumption*. La solución procesa 2,075,259 registros, aplica limpieza y validación concurrente, genera variables predictoras, construye la etiqueta *high_demand* y entrena una regresión logística binaria implementada sin librerías externas de Machine Learning.

El sistema fue diseñado como un pipeline modular que integra carga concurrente, preprocesamiento, feature engineering, entrenamiento paralelo, evaluación del modelo, análisis exploratorio, benchmarking y generación de evidencias reproducibles. La implementación utiliza goroutines, channels, WaitGroup, workers y reducción de resultados parciales, lo que permite demostrar procesamiento concurrente real sobre un volumen de datos superior a un millón de registros.

En la ejecución registrada se obtuvieron 2,049,280 registros válidos y 25,979 registros descartados, principalmente por valores faltantes. El modelo alcanzó un accuracy de 0.8559, una precisión de 0.8941, un recall de 0.4805 y un F1-score de 0.6251. Además, la función de pérdida disminuyó de 0.693147 a 0.358911, evidenciando convergencia durante el entrenamiento.

Los resultados del análisis exploratorio permitieron identificar patrones relevantes de consumo, especialmente picos de demanda entre las 19:00 y 21:00, mayor consumo promedio durante fines de semana y variaciones mensuales asociadas a estacionalidad. Estos hallazgos refuerzan la relación del proyecto con la sostenibilidad energética doméstica y dejan una base técnica preparada para futuras ampliaciones hacia servicios de predicción, procesamiento distribuido, almacenamiento de resultados y visualización web.

2. DESCRIPCIÓN DEL PROBLEMA Y MOTIVACIÓN

2.1 Contexto del problema

El consumo eléctrico doméstico constituye un componente relevante dentro de la sostenibilidad energética, debido a que los hogares generan patrones de demanda que pueden variar según la hora del día, el día de la semana, la estación del año y los hábitos de uso de los electrodomésticos. Cuando se presentan picos de consumo, estos pueden ocasionar mayores costos para los usuarios, incrementar la presión sobre la red eléctrica y reducir la eficiencia en el uso de los recursos energéticos.

En este contexto, el análisis de datos de consumo eléctrico permite identificar comportamientos asociados a periodos de alta demanda. Esta información puede ser utilizada como base para construir herramientas de apoyo a la toma de decisiones, generar alertas preventivas y orientar recomendaciones de eficiencia energética para usuarios domésticos.

2.2 Problema identificado

El problema abordado en este proyecto consiste en procesar un volumen masivo de lecturas eléctricas domésticas con el objetivo de identificar y predecir momentos de alta demanda energética. El dataset utilizado contiene más de dos millones de registros, por lo que su tratamiento requiere una estrategia eficiente de carga, limpieza, transformación y análisis.

La dificultad del problema no se limita únicamente a clasificar los registros como consumo normal o alta demanda. También implica procesar grandes volúmenes de información de manera concurrente, mantener trazabilidad sobre los registros válidos y descartados, generar variables relevantes para el modelo de Machine Learning y obtener evidencias experimentales que permitan evaluar tanto el rendimiento computacional como el desempeño predictivo del sistema.

2.3 Motivación

La motivación principal del proyecto es aplicar los conceptos centrales del curso en un problema real relacionado con sostenibilidad energética. El dataset seleccionado permite demostrar el uso de goroutines, channels, workers, sincronización, división de trabajo, reducción de resultados parciales y entrenamiento paralelo de un modelo de Machine Learning implementado íntegramente en Go.

Desde el punto de vista social, la predicción de alta demanda eléctrica puede contribuir a promover hábitos de consumo más eficientes. En futuras etapas del proyecto, los resultados del modelo podrán ser expuestos mediante una API y utilizados por una interfaz de consulta para alertar al usuario sobre momentos de posible alta demanda, apoyando así decisiones domésticas orientadas al ahorro energético y a la sostenibilidad.

2.4 Relación con sostenibilidad

El proyecto se relaciona directamente con la sostenibilidad energética porque busca transformar registros históricos de consumo eléctrico en información útil para la toma de decisiones. Al identificar patrones de alta demanda, el sistema puede servir como base para recomendaciones que ayuden a reducir picos de consumo y fomentar un uso más responsable de la energía en el hogar.

Entre los principales aportes esperados se encuentran la identificación de horarios críticos de consumo, el análisis de patrones asociados a la demanda doméstica, la preparación de un modelo reutilizable para futuras predicciones y la construcción de una base técnica que permita evolucionar hacia una solución distribuida con API, visualización e interacción con usuarios finales.

3. OBJETIVOS

3.1 Objetivo general

Desarrollar un sistema concurrente en Go para el análisis y predicción de alta demanda eléctrica doméstica, utilizando procesamiento paralelo de datos, limpieza y transformación de registros, generación de variables, entrenamiento de un modelo de Machine Learning y evaluación experimental reproducible, con una arquitectura preparada para evolucionar hacia una solución distribuida con API, persistencia y visualización en futuras entregas.

3.2 Objetivos específicos

1. Procesar el dataset Individual Household Electric Power Consumption, compuesto por más de dos millones de registros de consumo eléctrico doméstico.
2. Implementar un flujo de carga, limpieza y validación de datos en Go, utilizando goroutines, channels, WaitGroup y el patrón productor/consumidor.
3. Generar variables temporales y eléctricas que permitan representar patrones de consumo doméstico, tales como hora, día de la semana, mes, fin de semana, consumo reactivo, intensidad, submedidores y consumo no medido.
4. Definir una variable objetivo denominada `high_demand`, basada en el percentil 75 de `Global_active_power`, para transformar el problema en una tarea de clasificación binaria.
5. Entrenar un modelo de regresión logística binaria implementado en Go, sin utilizar librerías externas de Machine Learning, aplicando cálculo paralelo de gradientes sobre particiones del dataset.
6. Evaluar el desempeño del modelo mediante métricas de clasificación como accuracy, precision, recall, F1-score, matriz de confusión y evolución de la función de pérdida.
7. Medir el rendimiento computacional del sistema mediante benchmarks con distinta cantidad de workers, registrando tiempos de ejecución, registros procesados por segundo y comportamiento del procesamiento concurrente.
8. Persistir las evidencias generadas por el sistema, incluyendo dataset procesado, resumen de limpieza, métricas del modelo, modelo entrenado, predicciones de muestra, benchmarks y gráficos de análisis.
9. Diseñar una arquitectura modular que facilite la evolución del sistema hacia una solución distribuida en próximas entregas, incorporando API, cluster de nodos de Machine Learning, almacenamiento de resultados y visualización web.
10. Gestionar el desarrollo del proyecto mediante GitHub y Git Flow, manteniendo trazabilidad de ramas, issues, commits, milestones y evidencias de avance.

4. DESCRIPCIÓN DEL DATASET

4.1 Fuente del dataset

El dataset utilizado en este proyecto es Individual Household Electric Power Consumption, disponible en el UCI Machine Learning Repository. Este conjunto de datos contiene mediciones reales de consumo eléctrico doméstico y es utilizado frecuentemente para tareas de análisis de series temporales, predicción de consumo y evaluación de patrones energéticos.

Referencia del dataset:

Hebrail, G., & Berard, A. (2006). Individual Household Electric Power Consumption [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C58K54>

4.2 Descripción general

El archivo original contiene 2,075,259 lecturas de consumo eléctrico doméstico registradas con una frecuencia aproximada de un minuto. La información incluye variables temporales, potencia activa global, potencia reactiva global, voltaje, intensidad de corriente y tres submedidores asociados a consumos específicos dentro del hogar.

Para el proyecto se utilizó el archivo data/raw/household_power_consumption.txt. Durante la ejecución del sistema en Go, el dataset fue cargado, validado y limpiado de forma concurrente. Como resultado, se obtuvieron 2,049,280 registros válidos y se descartaron 25,979 registros por presentar valores faltantes o inconsistencias en el formato de los datos.

Característica	Valor
Archivo utilizado	data/raw/household_power_consumption.txt
Registros leídos	2,075,259
Registros válidos	2,049,280
Registros descartados	25,979
Separador	Punto y coma (;)
Frecuencia	Lecturas por minuto
Tema social	Sostenibilidad energética doméstica

4.3 Variables del dataset

El dataset original contiene nueve columnas principales. Las variables Date y Time permiten reconstruir la dimensión temporal de cada lectura, mientras que las variables eléctricas describen el comportamiento del consumo del hogar durante cada minuto registrado.

Variable	Descripción	Tipo
Date	Fecha de la medición.	Fecha
Time	Hora de la medición.	Tiempo
Global_active_power	Potencia activa global del hogar en kilowatts.	Continua
Global_reactive_power	Potencia reactiva global.	Continua
Voltage	Voltaje promedio por minuto.	Continua

Global_intensity	Intensidad de corriente global.	Continua
Sub_metering_1	Submedidor asociado principalmente a cocina.	Continua
Sub_metering_2	Submedidor asociado principalmente a lavandería.	Continua
Sub_metering_3	Submedidor asociado a calentador de agua y aire acondicionado.	Continua

4.4 Justificación de elección del dataset

La elección del dataset se justifica por su relación directa con el problema de sostenibilidad energética doméstica y por su volumen de información. Al contener más de dos millones de registros, el conjunto de datos permite demostrar la necesidad de aplicar procesamiento concurrente para acelerar la carga, limpieza, transformación y análisis de la información.

Además, se trata de un dataset real, público y documentado académicamente, lo que permite sustentar el trabajo con una fuente confiable. Sus variables temporales y eléctricas permiten construir características útiles para Machine Learning, así como formular una tarea de clasificación binaria orientada a detectar momentos de alta demanda energética.

5. LIMPIEZA Y ANÁLISIS DE DATOS

5.1 Problemas identificados en los datos

El dataset original presenta características que requieren una etapa de limpieza y validación antes de ser utilizado para el entrenamiento del modelo de Machine Learning. Los principales problemas identificados fueron los siguientes:

- Valores faltantes representados mediante el carácter ?.
- Variables numéricas almacenadas originalmente como texto.
- Fecha y hora separadas en columnas distintas.
- Necesidad de validar el formato temporal de cada registro.
- Posibles filas incompletas o con cantidad incorrecta de columnas.
- Necesidad de validar rangos eléctricos físicamente posibles, especialmente en variables como potencia, voltaje e intensidad.

Estos problemas justifican la implementación de una etapa de preprocesamiento robusta, ya que el entrenamiento del modelo requiere datos consistentes, numéricos y correctamente estructurados.

5.2 Estrategia de limpieza implementada

La limpieza fue implementada en Go dentro del paquete *internal/preprocessing*. Se aplicó una política conservadora: los registros incompletos, inconsistentes o no confiables fueron descartados en lugar de ser imputados. Esta decisión evita introducir valores artificiales en el entrenamiento inicial del proyecto y permite mantener trazabilidad clara sobre la calidad del dataset.

El proceso de limpieza considera las siguientes acciones:

- Ignorar la cabecera del archivo original.
- Descartar filas vacías o con columnas incompletas.
- Descartar registros con valores faltantes representados por ?.
- Convertir las columnas Date y Time en una representación temporal válida.
- Convertir las variables eléctricas a formato numérico.
- Validar rangos básicos de consumo, voltaje e intensidad.
- Procesar los registros mediante workers concurrentes.
- Acumular resultados parciales por worker.
- Reducir los resultados parciales en el coordinador principal.

Desde el punto de vista de programación concurrente, esta etapa utiliza goroutines, channels, WaitGroup y el patrón productor/consumidor. El lector del archivo actúa como productor de líneas, mientras que los workers consumen los registros, aplican validaciones y generan resultados parciales de limpieza.

5.3 Resultados de limpieza

La ejecución final del sistema se realizó sobre el archivo *data/raw/household_power_consumption.txt*, utilizando 8 workers. Como resultado, se procesaron 2,075,259 registros, de los cuales 2,049,280 fueron considerados válidos y 25,979 fueron descartados.

Métrica	Valor
Registros leídos	2,075,259
Registros válidos	2,049,280 (98.75%)
Registros descartados	25,979 (1.25%)
Principal motivo de descarte	missing_value
Workers utilizados	8
Tiempo de carga y limpieza	4822 ms
Filas procesadas por segundo	430354.55

El principal motivo de descarte fue *missing_value*, con 25,979 registros. Esto indica que el dataset es mayoritariamente consistente y que la limpieza conserva la mayor parte de la información original.

5.4 Variables derivadas

Después de la limpieza, se generaron variables adicionales para representar mejor los patrones temporales y eléctricos del consumo doméstico. Estas variables fueron creadas en Go durante la etapa de feature engineering y se almacenaron en el archivo *data/processed/features_high_demand.csv*.

Feature	Descripción
hour	Hora del día extraída del timestamp.
day_of_week	Día de la semana de la medición.
month	Mes de la medición.
is_weekend	Indicador binario que toma valor 1 si la medición corresponde a sábado o domingo.
voltage	Voltaje promedio por minuto.
global_reactive_power	Potencia reactiva global.
global_intensity	Intensidad eléctrica global.
sub_metering_1	Submedición asociada principalmente a cocina.
sub_metering_2	Submedición asociada principalmente a lavandería.
sub_metering_3	Submedición asociada a calentador de agua y aire acondicionado.
other_consumption	Consumo no medido directamente por los tres submedidores.
high_demand	Variable objetivo que indica si el registro corresponde a alta demanda eléctrica.

Estas variables permiten transformar el dataset original en una representación adecuada para clasificación binaria. En particular, las variables temporales ayudan a capturar patrones de consumo por hora, día y mes, mientras que las variables eléctricas representan el comportamiento energético del hogar.

5.5 Fórmula de consumo no medido

La variable *other_consumption* representa el consumo energético estimado que no está cubierto directamente por los tres submedidores principales. Se calcula mediante la siguiente fórmula:

$$other_consumption = (Global_active_power * 1000 / 60) - Sub_metering_1 - Sub_metering_2 - Sub_metering_3$$

En esta fórmula, *Global_active_power* se encuentra expresado en kilowatts. Por ello, se multiplica por 1000 para convertirlo a watts y se divide entre 60 para obtener el consumo equivalente por minuto. Luego se restan los consumos registrados por los tres submedidores.

Esta variable es relevante porque permite aproximar el consumo de otros equipos del hogar que no están registrados directamente en *Sub_metering_1*, *Sub_metering_2* o *Sub_metering_3*.

5.6 Estadísticas descriptivas principales

Como parte del análisis exploratorio, se calcularon estadísticas descriptivas sobre las principales variables eléctricas del dataset procesado. Estas métricas permiten comprender la distribución general de los datos antes del entrenamiento del modelo.

Variable	Media	Desv. est.	Min	P25	Mediana	P75	Max
global_active_power	1.0916	1.0573	0.0760	0.3080	0.6020	1.5280	11.1220
global_reactive_power	0.1237	0.1127	0.0000	0.0480	0.1000	0.1940	1.3900
voltage	240.8399	3.2400	223.2000	238.9900	241.0100	242.8900	254.1500
global_intensity	4.6278	4.4444	0.2000	1.4000	2.6000	6.4000	48.4000
sub_metering_1	1.1219	6.1530	0.0000	0.0000	0.0000	0.0000	88.0000
sub_metering_2	1.2985	5.8220	0.0000	0.0000	0.0000	1.0000	80.0000
sub_metering_3	6.4584	8.4372	0.0000	0.0000	1.0000	17.0000	31.0000
other_consumption	9.3147	9.5859	-2.4000	3.8000	5.5000	10.3667	124.8333

La potencia activa global presenta una media aproximada de 1.0916 kW, una mediana de 0.6020 kW y un percentil 75 de 1.5280 kW. Este último valor fue utilizado como umbral para definir la variable objetivo *high_demand*. De esta manera, los registros con *Global_active_power* mayor o igual a 1.5280 kW fueron clasificados como alta demanda, mientras que el resto fue clasificado como consumo normal.

También se observa que los submedidores presentan muchas mediciones en cero, lo cual indica que no todos los dispositivos o zonas del hogar están activos durante todo el periodo de medición. Por otro lado, *other_consumption* presenta valores altos en algunos casos, lo que sugiere la existencia de consumos no capturados por los tres submedidores principales.

5.7 Evidencias gráficas del análisis exploratorio

Las evidencias gráficas fueron generadas por el módulo de análisis implementado en Go. Estas figuras permiten interpretar visualmente la distribución del consumo eléctrico, los patrones temporales y la relación entre variables.

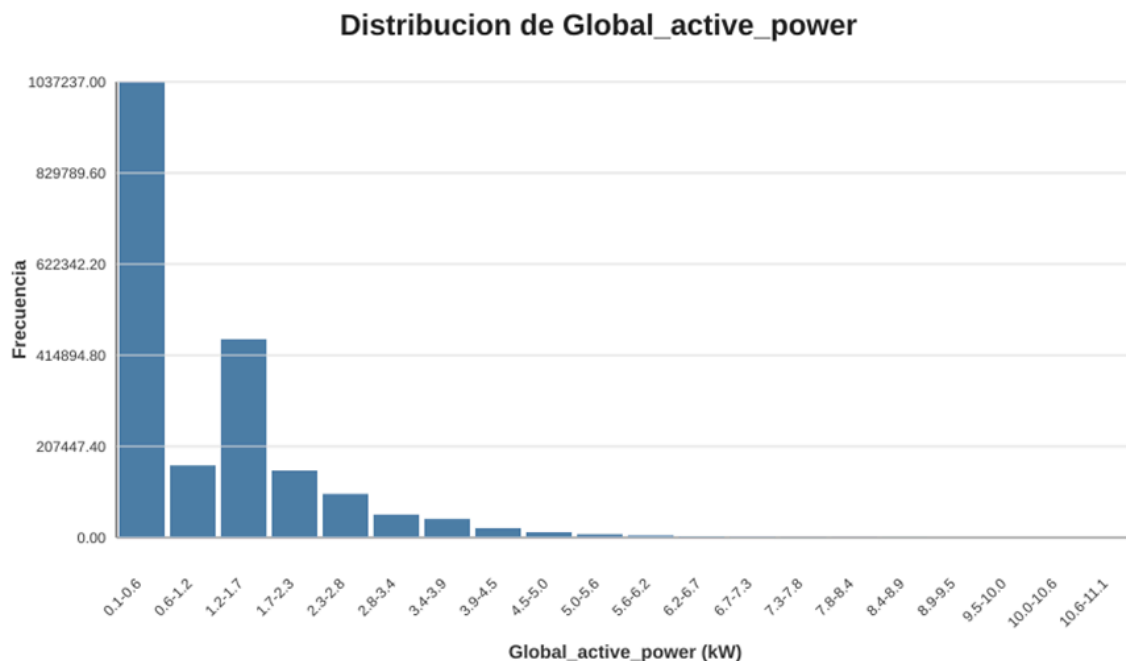


Figura 1. Distribución de Global_active_power.

La distribución de la potencia activa global muestra una alta concentración de registros en rangos bajos de consumo, especialmente entre 0.1 kW y 1.2 kW. También se observa una cola hacia la derecha, lo que indica que existen registros con consumos significativamente mayores, aunque con menor frecuencia. Este comportamiento justifica el uso del percentil 75 como criterio para identificar momentos de alta demanda, ya que permite separar los consumos más elevados del comportamiento habitual del hogar.



Figura 2. Consumo promedio por hora.

El consumo promedio por hora evidencia un patrón diario claro. Los valores más bajos se presentan durante la madrugada, principalmente entre las 02:00 y 05:00, cuando la actividad doméstica suele ser menor. A partir de las 06:00 se observa un incremento asociado al inicio de actividades del día. Los mayores niveles de consumo se concentran entre las 19:00 y 21:00, con un pico cercano a las 20:00. Este resultado es coherente con horarios de mayor presencia en el hogar y uso simultáneo de electrodomésticos.

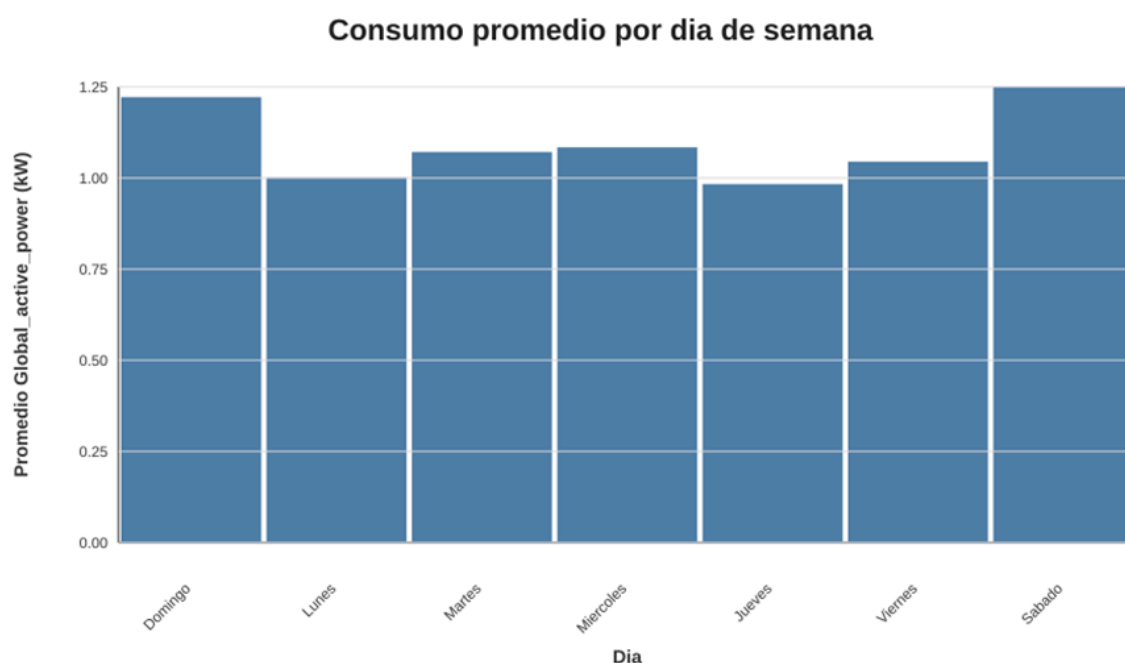


Figura 3. Consumo promedio por día de semana.

El análisis por día de semana muestra que sábado y domingo presentan los mayores niveles promedio de consumo. Esto puede estar asociado a una mayor permanencia de los habitantes en el hogar durante fines de semana. En contraste, los días laborales presentan valores ligeramente menores, aunque sin diferencias extremas. Esta evidencia respalda la inclusión de la variable *is_weekend* dentro del conjunto de características del modelo.

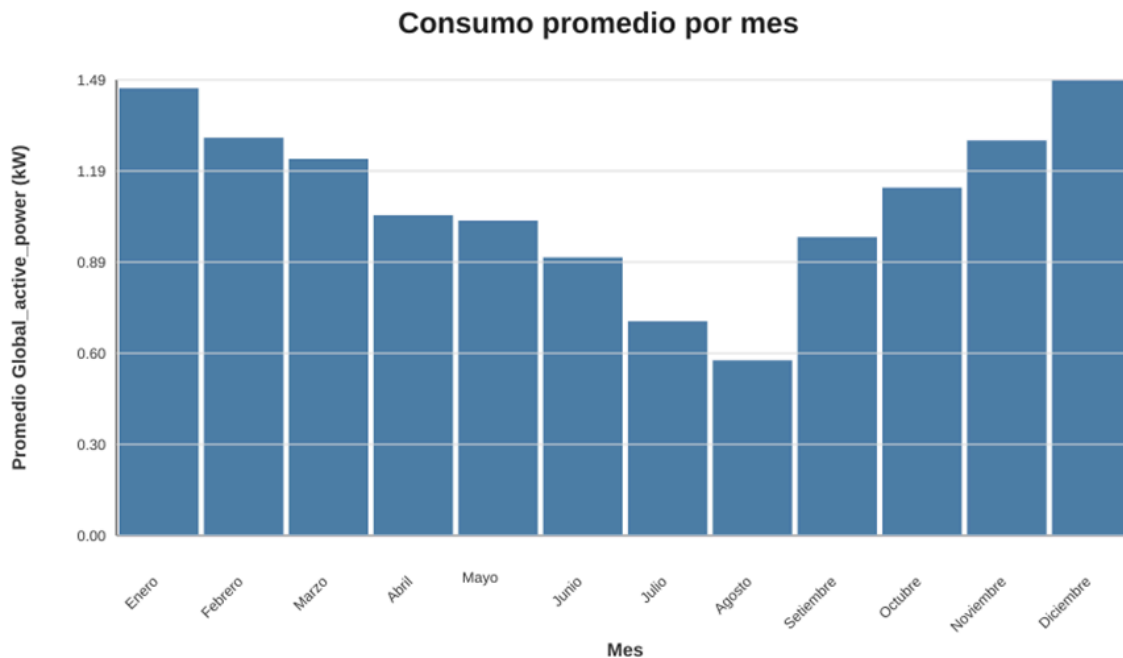


Figura 4. Consumo promedio por mes.

El análisis mensual muestra un comportamiento estacional. Los meses de diciembre, enero y febrero presentan consumos promedio más altos, mientras que julio y agosto registran los valores más bajos. Esta variación puede estar relacionada con condiciones climáticas, cambios de temperatura y hábitos de uso energético a lo largo del año. Por esta razón, la variable *month* fue incorporada como característica predictor

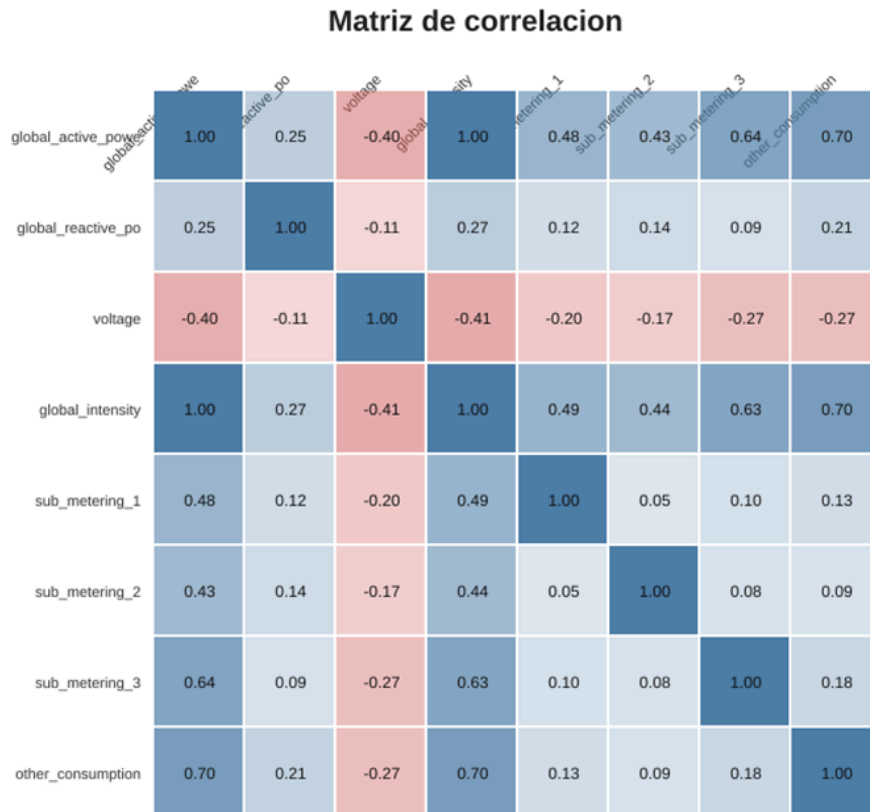


Figura 5. Matriz de correlación entre variables eléctricas.

La matriz de correlación evidencia relaciones importantes entre las variables del dataset. *Global_active_power* presenta una correlación casi perfecta con *global_intensity*, lo cual es esperable debido a la relación física entre potencia e intensidad eléctrica. También se observan correlaciones positivas relevantes entre *Global_active_power* y *other_consumption*, así como con *sub_metering_3*. Por otro lado, *voltage* presenta correlaciones negativas moderadas con variables de consumo, lo que indica que mayores niveles de demanda pueden estar asociados con ligeras disminuciones de voltaje.

5.8 Horas con mayor consumo promedio y tasa de alta demanda

El análisis por hora permitió identificar los periodos del día con mayor consumo promedio y mayor proporción de registros clasificados como alta demanda.

Hora	Registros	Consumo promedio	Tasa high_demand
20:00	85615	1.899064	55.32%
21:00	85529	1.877697	56.24%
19:00	85542	1.733335	49.08%

07:00	85284	1.502246	49.00%
08:00	85295	1.461016	38.42%

Los resultados muestran que las horas críticas se concentran principalmente entre las 19:00 y 21:00. En particular, a las 20:00 y 21:00 más del 55% de los registros corresponden a alta demanda. Este hallazgo es relevante para el enfoque de sostenibilidad del proyecto, ya que permite identificar franjas horarias donde podrían generarse recomendaciones futuras para reducir picos de consumo doméstico.

6. DISEÑO DEL MODELO DE MACHINE LEARNING

6.1 Tipo de problema

El problema fue formulado como una tarea de clasificación binaria. Cada registro limpio del dataset es clasificado en una de dos categorías: consumo normal o alta demanda eléctrica. Esta formulación permite transformar una variable originalmente continua, *Global_active_power*, en una variable objetivo interpretable y útil para aplicaciones de predicción, alerta y recomendación energética.

La elección de clasificación binaria es apropiada para el objetivo del proyecto, ya que permite identificar de manera directa aquellos momentos en los que el hogar presenta un consumo significativamente superior al comportamiento habitual. Desde la perspectiva de sostenibilidad, esta salida puede utilizarse posteriormente para alertar sobre horarios críticos y orientar decisiones de uso más eficiente de la energía.

6.2 Variable objetivo

La variable objetivo se definió a partir del percentil 75 de *Global_active_power*, calculado sobre el conjunto de registros válidos. El valor obtenido fue 1.528 kW, por lo que la variable *high_demand* se definió de la siguiente manera:

- $high_demand = 1$, si $Global_active_power \geq 1.528 \text{ kW}$
- $high_demand = 0$, en caso contrario

Esta decisión permite identificar el 25% superior de consumo eléctrico y transformar el problema en una clasificación binaria orientada a detectar momentos de alta demanda.

Clase	Cantidad	Proporción
Demanda normal	1,535,854	74.95%
Alta demanda	513,426	25.05%

La distribución de clases muestra que el problema presenta un desbalance moderado, ya que la clase de alta demanda representa aproximadamente una cuarta parte del total. Esta característica debe tenerse en cuenta al interpretar métricas como accuracy, precision y recall.

6.3 Variables predictoras

El modelo utiliza 11 variables predictoras, las cuales fueron normalizadas mediante el método min-max por columna sobre el conjunto limpio. Esta normalización permite que las variables trabajen en escalas comparables, lo cual favorece la estabilidad del entrenamiento y mejora la convergencia del algoritmo de optimización.

Las variables predictoras utilizadas son las siguientes:

- hour
- day_of_week
- month
- is_weekend
- voltage
- global_reactive_power
- global_intensity
- sub_metering_1
- sub_metering_2
- sub_metering_3
- other_consumption

Estas variables combinan información temporal y eléctrica. Las variables temporales permiten capturar patrones diarios, semanales y mensuales de consumo, mientras que las variables eléctricas describen el estado energético del hogar en cada lectura.

6.4 Modelo seleccionado

Se implementó una regresión logística binaria propia en Go, sin utilizar librerías externas de Machine Learning. Esta elección resulta adecuada para el proyecto porque permite resolver una tarea de clasificación binaria de forma interpretable, eficiente y compatible con una arquitectura concurrente.

En primer lugar, la regresión logística es un modelo apropiado para estimar la probabilidad de pertenencia a una clase, en este caso la probabilidad de que un registro corresponda a alta demanda eléctrica. En segundo lugar, su implementación permite controlar de manera explícita los elementos centrales del entrenamiento supervisado: función sigmoide, cálculo de probabilidades, función de pérdida, cálculo de gradientes y actualización iterativa de pesos.

Además, este modelo se adapta bien al enfoque del curso, ya que el cálculo de gradientes puede dividirse entre varios workers. De esta manera, cada worker procesa una partición del dataset, calcula gradientes parciales y los envía al coordinador principal. Posteriormente, el coordinador combina los resultados parciales y actualiza los parámetros globales del modelo. Este flujo permite demostrar paralelización real del entrenamiento, sincronización de resultados y reducción de cálculos parciales.

El uso de una implementación propia también facilita la reutilización del modelo en las siguientes etapas del sistema, debido a que los pesos, parámetros de normalización y configuración del entrenamiento pueden almacenarse en un archivo JSON y ser cargados posteriormente por una API o por nodos de cómputo distribuidos.

6.5 Métricas de evaluación

Para evaluar el desempeño del modelo se utilizaron las métricas de clasificación más relevantes: accuracy, precision, recall, F1-score y la evolución de la función de pérdida durante el entrenamiento.

Métrica	Valor	Interpretación
Accuracy	0.8559	Proporción total de predicciones correctas.
Precision	0.8941	Cuando el modelo predice alta demanda, suele acertar con alta confiabilidad.
Recall	0.4805	El modelo detecta una parte relevante de los casos de alta demanda, aunque aún puede mejorar.
F1-score	0.6251	Balance entre precision y recall.
Loss inicial	0.693147	Punto de partida del entrenamiento.
Loss final	0.358911	La disminución evidencia aprendizaje.
Normalización	min-max por columna sobre el conjunto limpio	Escalado aplicado a las variables predictoras.

Estas métricas muestran que el modelo alcanza un buen desempeño general, con una accuracy de 0.8559 y una precision de 0.8941. Esto significa que, cuando el clasificador predice un caso de alta demanda, la probabilidad de que esa predicción sea correcta es alta. Sin embargo, el recall de 0.4805 indica que todavía existe una proporción importante de casos reales de alta demanda que no son detectados por el modelo.

Matriz de confusión:

La matriz de confusión permite descomponer el comportamiento del clasificador según aciertos y errores por clase.

	Predicho normal	Predicho alta demanda
Real normal	301547	5829
Real alta demanda	53242	49238

Matriz de confusion

		Pred. 0	Pred. 1
Valor real	Real 0	TN 301547	FP 5829
	Real 1	FN 53242	TP 49238
		Prediccion	

Figura 6. Matriz de confusión del clasificador binario.

A partir de esta matriz se observan varios aspectos importantes:

- El modelo identifica correctamente 301,547 casos de demanda normal (true negatives).
- Predice correctamente 49,238 casos de alta demanda (true positives).
- Solo 5,829 casos normales fueron clasificados erróneamente como alta demanda (false positives), lo que explica la alta precision obtenida.
- Sin embargo, 53,242 casos reales de alta demanda fueron clasificados como demanda normal (false negatives), lo que reduce el recall.

En términos prácticos, esto indica que el modelo es conservador: evita muchas falsas alarmas, pero todavía deja pasar una cantidad considerable de eventos reales de alta demanda. En esta versión funcional del sistema, este comportamiento es aceptable, ya que el objetivo principal no es maximizar el rendimiento predictivo absoluto, sino demostrar una implementación funcional, concurrente y experimentalmente trazable del pipeline de Machine Learning en Go.

Curva de pérdida durante el entrenamiento:

La evolución de la función de pérdida permite verificar si el modelo está aprendiendo durante las iteraciones de entrenamiento.

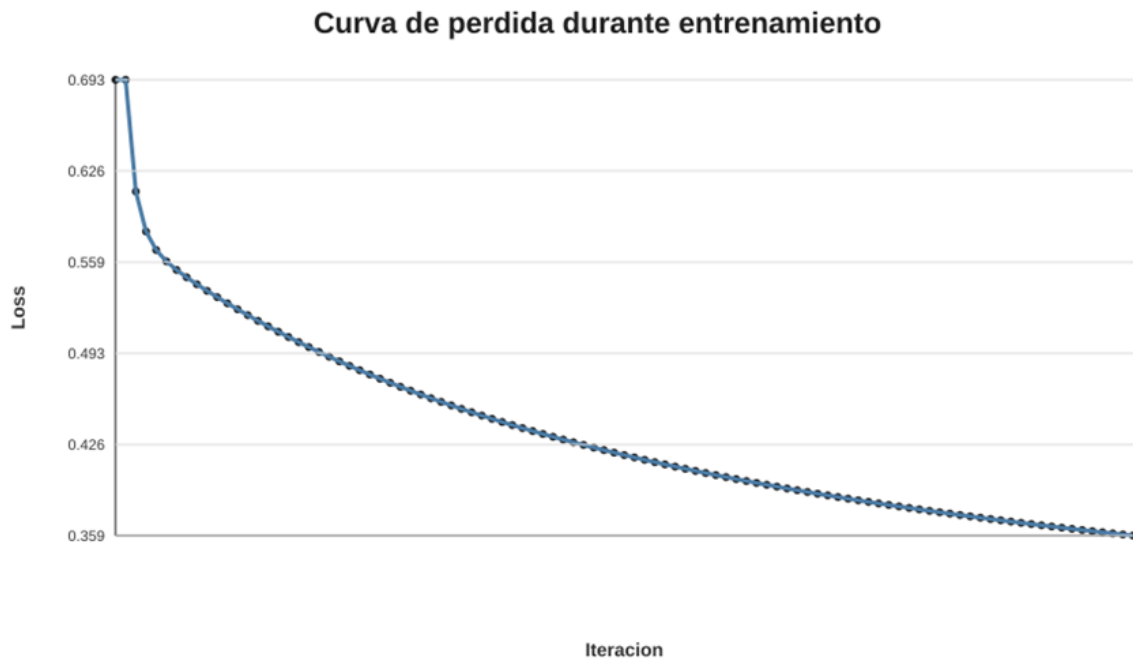


Figura 7. Curva de pérdida durante el entrenamiento.

La curva muestra una disminución sostenida del loss, desde un valor inicial de 0.693147 hasta un valor final de 0.358911. El valor inicial cercano a 0.693 es consistente con el comportamiento esperado de una regresión logística al comenzar con pesos cercanos a cero, donde la probabilidad estimada se aproxima a 0.5 para ambas clases.

A medida que avanzan las iteraciones, el valor de la pérdida decrece de manera estable, sin oscilaciones severas ni divergencia. Esto evidencia que el proceso de optimización converge adecuadamente y que la combinación de tasa de aprendizaje, número de iteraciones y normalización de variables fue suficiente para entrenar el modelo de forma efectiva.

La forma de la curva también sugiere que el aprendizaje fue más rápido en las primeras iteraciones y luego se volvió más gradual, lo cual es habitual en procesos de optimización iterativa. Este comportamiento es consistente con un entrenamiento estable y respalda la validez de la implementación realizada en Go.

Interpretación del desempeño:

En conjunto, los resultados muestran que el modelo presenta un comportamiento consistente con los objetivos del sistema. La precisión obtenida es alta, lo que indica que las predicciones positivas son confiables: cuando el modelo clasifica un registro como alta demanda, generalmente acierta. Esto es importante para evitar alertas innecesarias o recomendaciones incorrectas al usuario final.

El recall obtenido es medio, lo que indica que el modelo todavía no detecta todos los casos reales de alta demanda. En otras palabras, el clasificador es conservador: prefiere reducir los falsos positivos, aunque esto implique dejar pasar algunos eventos reales de alta demanda.

Este comportamiento puede ser aceptable en una primera versión funcional del sistema, pero también identifica una oportunidad clara de mejora.

La curva de pérdida muestra una disminución sostenida durante el entrenamiento, lo que evidencia que el proceso de optimización converge adecuadamente y que el modelo aprende patrones presentes en los datos. La reducción del loss desde 0.693147 hasta 0.358911 respalda la validez del entrenamiento implementado.

Como líneas futuras de mejora, podrían evaluarse estrategias como el ajuste del umbral de decisión, la ponderación de clases, el balanceo del dataset, la incorporación de nuevas variables temporales o la comparación con modelos alternativos. No obstante, la regresión logística implementada cumple adecuadamente con el objetivo de servir como base funcional, interpretable y reutilizable para la evolución posterior del sistema.

7. PARALELIZACIÓN DEL CÁLCULO

7.1 Justificación de paralelización

El dataset utilizado contiene más de dos millones de registros, por lo que su procesamiento secuencial puede afectar el rendimiento del sistema y limitar la escalabilidad de la solución. Debido a este volumen de datos, se implementó una arquitectura concurrente en Go que divide el trabajo en unidades independientes y permite que múltiples goroutines procesen información en paralelo.

La paralelización se justifica por dos razones principales. En primer lugar, permite acelerar etapas costosas como la carga, limpieza, validación y entrenamiento del modelo. En segundo lugar, responde al objetivo central del sistema: construir una solución capaz de procesar grandes volúmenes de datos de forma eficiente, manteniendo sincronización, trazabilidad y control sobre los resultados generados.

Para ello, se utilizaron mecanismos propios de Go como goroutines, channels y WaitGroup. Además, se aplicó reducción de resultados parciales para evitar actualizaciones concurrentes directas sobre estructuras compartidas, reduciendo así el riesgo de condiciones de carrera.

7.2 Etapas paralelizadas

La solución paraleliza principalmente dos etapas del flujo de trabajo: la carga y limpieza de datos, y el entrenamiento del modelo de Machine Learning.

Carga y limpieza concurrente:

El módulo *internal/loader/concurrent_loader.go* implementa un patrón productor/consumidor. Una goroutine lectora recorre el archivo original y envía las filas crudas a un channel de trabajos. Luego, varios workers consumen dichas filas, ejecutan el parseo, validan los campos, convierten los valores numéricos y acumulan resultados parciales.

Cada worker trabaja de forma independiente sobre una porción del flujo de datos. En lugar de actualizar contadores globales directamente, los workers generan resultados parciales que posteriormente son consolidados por el coordinador. Esta decisión evita compartir estado mutable durante la fase de procesamiento y reduce el riesgo de errores de sincronización.

El flujo aplicado en esta etapa puede resumirse de la siguiente manera:

Goroutine lectora → Channel de filas raw → Workers de limpieza → Resultados parciales → Reducción final

Entrenamiento paralelo del modelo:

El módulo *internal/ml/logistic_regression.go* paraleliza el entrenamiento de la regresión logística. Para cada iteración, el dataset de entrenamiento se divide en particiones. Cada goroutine procesa una partición y calcula gradientes parciales asociados a los pesos, el sesgo y la pérdida.

Después de que los workers terminan su cálculo, el coordinador combina los gradientes parciales y actualiza los parámetros globales del modelo. Esta estrategia permite mantener la actualización de pesos en un punto controlado del programa, evitando que varios workers modifiquen simultáneamente los mismos parámetros.

El flujo aplicado en esta etapa puede resumirse de la siguiente manera:

Dataset de entrenamiento → Particionado por chunks → Cálculo de gradientes parciales → Reducción de gradientes → Actualización de pesos

7.3 Patrones de concurrencia utilizados

La solución utiliza distintos patrones de concurrencia que se complementan dentro del pipeline general del sistema.

Patrón	Aplicación en el proyecto	Beneficio
Productor/consumidor	Lectura del archivo y distribución de filas hacia workers.	Permite desacoplar lectura y procesamiento.
Worker pool	Limpieza y validación concurrente de registros.	Distribuye el trabajo entre varias goroutines.
Channels	Comunicación entre productor, workers y coordinador.	Evita acoplamiento directo entre componentes.

WaitGroup	Espera controlada la finalización de goroutines.	Garantiza sincronización antes de cerrar canales o reducir resultados.
Reducción de parciales	Consolidación de registros válidos, descartes y gradientes.	Evita modificar el estado global desde múltiples workers.
Particionado por chunks	Entrenamiento paralelo del modelo ML.	Divide el cálculo de gradientes sobre subconjuntos del dataset.

Patrones de concurrencia implementados

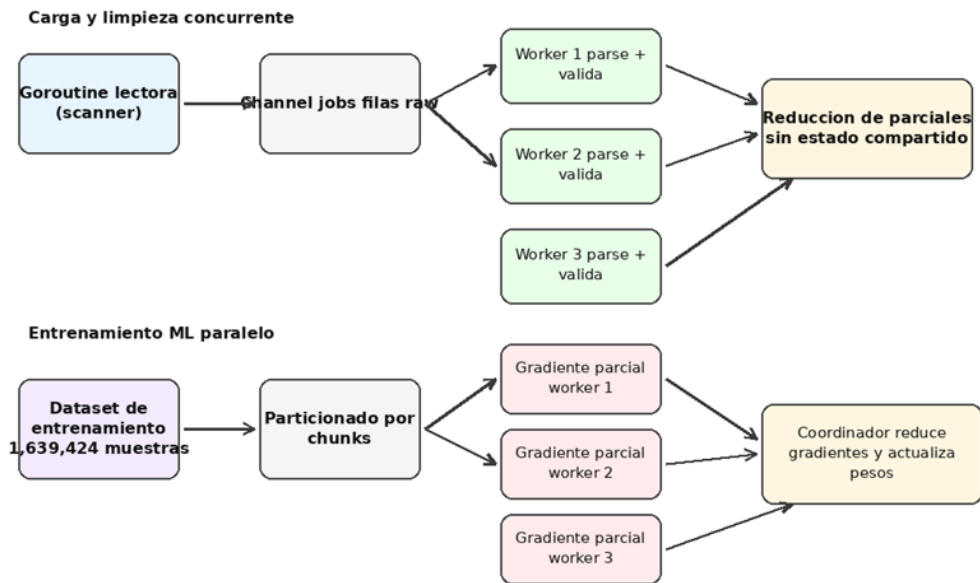


Figura 8. Patrones de concurrencia utilizados en carga/limpieza y entrenamiento ML.

La figura muestra dos flujos principales. En la parte superior se representa la carga y limpieza concurrente, donde una goroutine lectora envía filas crudas a un channel y varios workers aplican parseo y validación. En la parte inferior se representa el entrenamiento paralelo, donde el dataset de entrenamiento se divide en particiones y cada worker calcula gradientes parciales. En ambos casos, el coordinador central realiza la reducción final de resultados, lo que permite mantener control sobre la sincronización y evitar condiciones de carrera.

7.4 Métricas de rendimiento de la ejecución concurrente

La ejecución final del sistema se realizó utilizando 8 workers. Los tiempos obtenidos permiten evidenciar el comportamiento del pipeline completo, desde la carga y limpieza de datos hasta el entrenamiento del modelo.

Métrica	Valor
Workers	8
Registros procesados	2,075,259
Tiempo de carga y limpieza	2207 ms
Tiempo de feature engineering	323 ms

Tiempo de entrenamiento	6017 ms
Tiempo total	12928 ms
Registros por segundo	160515.78

Estos resultados muestran que el sistema procesa más de dos millones de registros en un tiempo total aproximado de 12.9 segundos. La etapa de entrenamiento es la más costosa del pipeline, debido a que ejecuta múltiples iteraciones de optimización y cálculo de gradientes sobre el conjunto de entrenamiento. En contraste, la etapa de feature engineering presenta un tiempo reducido, lo que indica que la transformación de variables no representa el principal cuello de botella del sistema.

Los tiempos registrados pueden variar según el hardware, la carga del sistema operativo y la configuración de ejecución. Sin embargo, sirven como evidencia experimental de que la solución no solo implementa concurrencia, sino que también mide su rendimiento y conserva resultados reproducibles para análisis posterior.

7.5 Consideraciones sobre seguridad concurrente

Para reducir riesgos de condiciones de carrera, el diseño evita que múltiples goroutines actualicen simultáneamente las mismas estructuras globales. En la carga y limpieza, cada worker genera resultados locales que luego son reducidos por el coordinador. En el entrenamiento, cada worker calcula gradientes parciales, pero la actualización de pesos se realiza únicamente después de consolidar los resultados.

Esta estrategia permite separar claramente el cálculo paralelo de la actualización de estado global. Como resultado, el sistema mantiene una estructura más segura, trazable y fácil de extender hacia una arquitectura distribuida, donde los workers podrían evolucionar posteriormente hacia nodos de cómputo independientes.

8. ARQUITECTURA FUNCIONAL DEL SISTEMA

La arquitectura del sistema fue diseñada como un pipeline modular implementado en Go. Cada módulo cumple una responsabilidad específica dentro del flujo de procesamiento, desde la lectura del dataset original hasta la generación de resultados, métricas, evidencias y gráficos. Esta organización permite mantener separación de responsabilidades, facilitar la revisión del código y preparar la solución para futuras extensiones.

La arquitectura sigue un flujo secuencial a nivel de etapas, pero utiliza concurrencia interna en los procesos más costosos, especialmente durante la carga, limpieza y entrenamiento del modelo. De esta manera, el sistema combina modularidad con procesamiento paralelo.

Arquitectura funcional PC3: pipeline concurrente y ML en Go

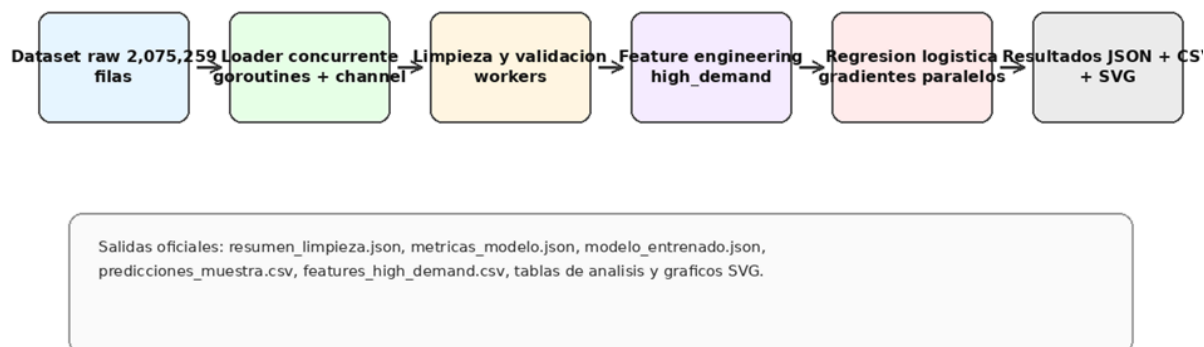


Figura 9. Arquitectura funcional del pipeline concurrente y ML en Go.

La figura representa el recorrido completo de los datos. El proceso inicia con el dataset original, compuesto por 2,075,259 filas. Luego, el loader concurrente distribuye el trabajo mediante goroutines y channels. Posteriormente, los registros son limpiados y validados por workers, transformados mediante feature engineering, usados para entrenar una regresión logística con gradientes paralelos y finalmente almacenados como resultados en formatos JSON, CSV y SVG.

8.1 Componentes técnicos

Componente	Ruta	Responsabilidad
Entrada del sistema	cmd/pc3/main.go	Coordina pipeline, parametros, salidas y persistencia de evidencias.
Carga concurrente	internal/loader	Lee el dataset y distribuye filas hacia workers mediante channels.
Preprocesamiento	internal/preprocessing	Parsea fechas, convierte variables y descarta registros invalidos.
Feature engineering	internal/features	Genera variables predictoras y etiqueta high_demand.
Machine Learning	internal/ml	Entrena regresion logistica con gradientes paralelos.
Metricas	internal/metrics	Calcula accuracy, precision, recall, F1 y rendimiento.
Analisis	internal/analysis	Genera tablas estadisticas y graficos para el informe.
Visualizacion	internal/visualization	Produce graficos SVG desde Go.

8.2 Flujo de ejecucion

El flujo de ejecución del sistema se organiza en las siguientes etapas:

- Lectura concurrente del archivo raw *household_power_consumption.txt*.
- Distribución de filas crudas hacia workers mediante channels.
- Limpieza y validación de registros.
- Ordenamiento determinista de los registros válidos según su línea original.
- Generación de variables predictoras mediante feature engineering.
- Cálculo del percentil 75 de *Global_active_power*.
- Creación de la variable objetivo *high_demand*.
- Normalización min-max de las variables predictoras.
- División del dataset en conjuntos de entrenamiento y prueba.
- Entrenamiento paralelo del modelo de regresión logística.
- Evaluación del modelo sobre el conjunto de prueba.
- Guardado del modelo entrenado, métricas, predicciones y dataset procesado.
- Generación de análisis exploratorio y gráficos en Go.
- Persistencia de evidencias en carpetas de resultados.

Este flujo garantiza que cada ejecución del sistema produzca archivos verificables y reutilizables, tales como *resumen_limpieza.json*, *metricas_modelo.json*, *modelo_entrenado.json*, *predicciones_muestra.csv*, *features_high_demand.csv*, tablas de análisis y gráficos SVG.

8.3 Preparación para evolución futura del sistema

La arquitectura fue diseñada para que los resultados generados puedan ser reutilizados en análisis posteriores o en una versión ampliada del sistema. El archivo *modelo_entrenado.json* almacena los pesos, parámetros de normalización y configuración del modelo, lo que permite cargarlo nuevamente sin repetir todo el entrenamiento.

Asimismo, el dataset procesado y las métricas exportadas en formatos JSON y CSV facilitan la integración con otros componentes, como servicios de consulta, dashboards, almacenamiento persistente o nodos de cómputo especializados. De esta manera, el sistema no solo resuelve el flujo de análisis y entrenamiento, sino que también deja una base técnica ordenada para escalar hacia escenarios de predicción, visualización y procesamiento distribuido.

9. EVIDENCIAS DE IMPLEMENTACIÓN

Esta sección presenta las evidencias generadas durante la implementación y ejecución del sistema. El objetivo no es mostrar únicamente la versión final del programa, sino documentar el proceso completo de validación: compilación, ejecución del pipeline, generación de resultados, análisis exploratorio, gráficos, benchmarks y estructura de archivos producidos.

Las evidencias permiten demostrar que el sistema fue implementado en Go, que procesa el dataset completo, que aplica concurrencia real y que genera resultados reproducibles para su revisión.

9.1 Evidencia de ejecución final

La ejecución final del sistema se realizó sobre el archivo *data/raw/household_power_consumption.txt*, utilizando 8 workers. Esta ejecución permitió validar el flujo completo del proyecto: lectura concurrente, limpieza, feature engineering, entrenamiento paralelo, evaluación del modelo y persistencia de evidencias.

```
Ejecucion PC3 completada
Registros leidos: 2075259
Registros validos: 2049280
Registros descartados: 25979
Workers usados: 8
Muestras entrenamiento: 1639424
Muestras prueba: 409856
Loss inicial: 0.693147 | Loss final: 0.358911
Accuracy: 0.8559 | Precision: 0.8941 | Recall: 0.4805 | F1: 0.6251
Modelo guardado en: results\modelo_entrenado.json
Dataset procesado guardado en: data\processed\features_high_demand.csv
Resultados generados en: results
```

Figura 10. Salida de consola de la ejecución final registrada.

La salida de consola evidencia que el sistema procesó 2,075,259 registros, obtuvo 2,049,280 registros válidos y descartó 25,979 registros. Además, muestra la división entre muestras de entrenamiento y prueba, los valores de loss inicial y final, las métricas del modelo y las rutas donde se almacenaron los resultados principales.

Esta evidencia es importante porque demuestra que el pipeline completo se ejecuta correctamente sobre el dataset real y no únicamente sobre una muestra reducida.

9.2 Evidencias generadas por el sistema

Durante la ejecución se generaron archivos en formato JSON, CSV y SVG. Estos archivos permiten sustentar los resultados del proyecto y reutilizar la información para análisis posteriores.

Archivo generado	Ubicación	Uso en el informe
resumen_limpieza.json	results/final_run	Contiene conteos de registros leídos, válidos, descartados y motivos de descarte.
metricas_modelo.json	results/final_run	Contiene métricas del modelo, matriz de confusión, loss inicial, loss final e historial de entrenamiento.

modelo_entrenado.json	results/final_run	Almacena el modelo entrenado, pesos, sesgo y parámetros necesarios para reutilización.
predicciones_muestra.csv	results/final_run	Presenta una muestra de predicciones generadas por el clasificador.
features_high_demand.csv	data/processed	Dataset procesado con variables predictoras y variable objetivo.
Tablas de análisis	results/analysis	Contiene estadísticas descriptivas, correlaciones y agregaciones por hora, día y mes.
Gráficos SVG	results/figures	Contiene las figuras utilizadas como evidencias visuales del análisis y evaluación.
Resultados de benchmark	results/benchmarks	Contiene ejecuciones comparativas con distinta cantidad de workers.

Estos archivos permiten verificar que el sistema no solo imprime resultados en consola, sino que conserva evidencias persistentes para auditoría, revisión académica y reproducción experimental.

9.3 Estructura de evidencias del proyecto

La estructura del proyecto fue organizada para separar el código fuente, los datos originales, los datos procesados, los resultados finales, los análisis, las figuras y los scripts de reproducción.

Estructura de evidencias del proyecto

```

1ACC0065_PC3_U202222148/
├─ cmd/pc3, cmd/pc3-analysis, cmd/pc3-benchmark
├─ internal/loader, preprocessing, features, ml, metrics, analysis, visualization
├─ data/raw/household_power_consumption.txt
├─ data/processed/features_high_demand.csv
├─ results/final_run/ JSON + modelo + predicciones
├─ results/analysis/ tablas CSV/JSON
├─ results/figures/ graficos SVG
├─ scripts/ PowerShell y Bash
├─ docker/Dockerfile
└─ docs/ informe y guias de ejecucion

```

Figura 11. Estructura de evidencias del proyecto.

La estructura de carpetas permite identificar claramente el flujo de trabajo:

- cmd/pc3: contiene el ejecutable principal del pipeline.
- cmd/pc3-analysis: genera análisis estadístico y gráficos.
- cmd/pc3-benchmark: ejecuta pruebas comparativas de rendimiento.

- internal: contiene los paquetes internos de carga, limpieza, features, Machine Learning, métricas, análisis y visualización.
- data/raw: almacena el dataset original.
- data/processed: almacena el dataset transformado.
- results/final_run: almacena la ejecución final.
- results/analysis: almacena tablas y resultados de análisis exploratorio.
- results/figures: almacena gráficos generados en Go.
- results/benchmarks: almacena pruebas de rendimiento por cantidad de workers.
- scripts: contiene comandos automatizados para PowerShell y Bash.
- docs: contiene documentación y guías de ejecución.

Esta organización permite demostrar orden técnico, trazabilidad y separación de responsabilidades dentro del proyecto.

9.4 Evidencias del proceso de prueba y validación

Además de la ejecución final, el proyecto conserva evidencias del proceso experimental. Este proceso incluye pruebas de compilación, ejecuciones del pipeline, generación de análisis, generación de figuras y benchmarks.

Las principales etapas de validación fueron las siguientes:

Etapas	Evidencia	Propósito
Compilación del proyecto	go test ./...	Verificar que todos los paquetes compilan correctamente.
Ejecución principal	cmd/pc3	Validar el pipeline completo con el dataset real.
Limpieza de datos	resumen_limpieza.json	Comprobar registros válidos, descartados y motivos de descarte.
Feature engineering	features_high_demand.csv	Confirmar la generación de variables predictoras y target.
Entrenamiento ML	metricas_modelo.json y modelo_entrenado.json	Evidenciar entrenamiento, evaluación y persistencia del modelo.
Análisis exploratorio	results/analysis	Generar estadísticas, correlaciones y agregaciones.
Visualización	results/figures	Generar gráficos SVG desde Go.
Benchmark concurrente	results/benchmarks	Comparar rendimiento con diferente cantidad de workers.

Esta secuencia permite demostrar que el proyecto fue desarrollado y validado de forma progresiva, no únicamente ejecutado al final. Cada etapa produce archivos que pueden ser revisados de manera independiente.

9.5 Evidencias de benchmark y rendimiento

Para evaluar el impacto del procesamiento concurrente, se implementó un comando específico de benchmark en Go. Este comando ejecuta el pipeline con diferentes cantidades de workers y registra métricas de rendimiento.

Las pruebas de benchmark permiten comparar configuraciones como:

- 1 worker
- 2 workers
- 4 workers
- 8 workers

El objetivo de estas pruebas es observar cómo varían los tiempos de ejecución y la tasa de procesamiento cuando se modifica el nivel de concurrencia. Los resultados se almacenan en `results/benchmarks` y pueden ser usados para construir tablas comparativas de tiempo total, registros procesados por segundo, speedup y eficiencia.

Evidencia	Ubicación	Descripción
Benchmark con 1 worker	results/benchmarks	Ejecución base con menor nivel de concurrencia.
Benchmark con 2 worker	results/benchmarks	Evaluación con paralelismo moderado.
Benchmark con 4 worker	results/benchmarks	Evaluación con mayor distribución de trabajo.
Benchmark con 8 worker	results/benchmarks	Configuración usada para la ejecución final.
Gráfico de benchmark	results/benchmarks	Visualización comparativa del rendimiento.

Estas evidencias son relevantes porque muestran que el proyecto no solo utiliza goroutines y channels, sino que también mide experimentalmente el comportamiento del sistema.

9.6 Comandos de reproducción

Para asegurar que el proyecto pueda ser revisado y ejecutado nuevamente, se documentaron los comandos principales de reproducción.

Compilación y pruebas generales:

```
go test ./...
```

Ejecución completa del pipeline:

```
go run ./cmd/pc3 -input data/raw/household_power_consumption.txt -workers 8 -out results/final_run -processed-out data/processed -iterations 300 -lr 1.0
```

Generación de análisis exploratorio y gráficos:

```
go run ./cmd/pc3-analysis -input data/raw/household_power_consumption.txt -workers 8 -out results/analysis -figures results/figures -metrics results/final_run/metricas_modelo.json
```

Ejecución de benchmark:

```
go run ./cmd/pc3-benchmark -input data/raw/household_power_consumption.txt -out results/benchmarks -figures results/figures -workers 1,2,4,8 -iterations 300 -lr 1.0 -run=true
```

También se incluyeron scripts equivalentes en la carpeta scripts para PowerShell y Bash. Esto permite reproducir la ejecución, el análisis y las evidencias sin depender de herramientas externas de análisis de datos.

9.7 Evidencia de desarrollo iterativo

Durante el desarrollo se realizaron varias validaciones incrementales antes de consolidar la ejecución final. Primero se verificó la compilación del proyecto, luego se ejecutó el pipeline con datos de prueba, posteriormente se procesó el dataset completo y finalmente se incorporaron análisis, gráficos y benchmarks.

Este proceso permitió detectar y corregir aspectos como parámetros de ejecución, organización de carpetas, persistencia de resultados, generación de evidencias y comandos reproducibles. Como resultado, el proyecto conserva una estructura que permite revisar tanto el resultado final como el camino experimental seguido para obtenerlo.

El enfoque seguido demuestra un desarrollo progresivo: primero se construyó un pipeline funcional, luego se validó con el dataset completo, después se agregaron métricas y persistencia, y finalmente se incorporaron análisis estadísticos, visualización y benchmarking concurrente.

10. GESTIÓN DEL PROYECTO EN GITHUB

El proyecto se gestionó mediante GitHub como repositorio remoto y Git como sistema de control de versiones. Esta decisión permitió mantener trazabilidad sobre los cambios realizados en el código fuente, la documentación, los scripts de ejecución y la estructura de evidencias generadas durante el desarrollo.

El uso de versionamiento resulta importante porque permite identificar la evolución del proyecto desde la selección del dataset hasta la implementación del pipeline concurrente, la limpieza de datos, la generación de variables, el entrenamiento paralelo del modelo y la producción de resultados experimentales.

Elemento	Evidencia
Repositorio remoto	https://github.com/Omar362267/TB2-Programaci-n-Concurrente-.git

Rama principal	main
Lenguaje principal	go
Commits relevantes	feat: implement concurrent preprocessing and parallel ML training; Analisis y limpieza de datos; chore: Actualizacion de gitignore.
Componentes versionados	Código fuente, scripts, documentación, estructura de carpetas y evidencias del proyecto.
Estrategia de ramas	Mantener main como rama estable y utilizar ramas feature/* para nuevas funcionalidades.
Flujo de trabajo	Desarrollo incremental mediante commits asociados a carga de datos, limpieza, concurrencia, Machine Learning, análisis y documentación.

La rama main se mantiene como versión estable del proyecto. Para nuevas funcionalidades o ampliaciones del sistema, se recomienda continuar utilizando ramas de tipo feature/*, por ejemplo feature/api-predictions, feature/distributed-nodes, feature-dashboard o feature-benchmark-improvements. Este enfoque permite conservar una línea principal estable y desarrollar nuevas capacidades sin afectar directamente la versión funcional.

Como evidencia de gestión del proyecto, se deben anexar capturas del repositorio remoto, historial de commits, estructura de ramas, issues, milestones y tablero Kanban. Estas evidencias permiten demostrar organización, trazabilidad, colaboración y seguimiento del avance técnico.

En el contexto del proyecto, GitHub cumple tres funciones principales:

1. Centralizar el código fuente y la documentación.
2. Registrar la evolución del desarrollo mediante commits.
3. Facilitar la continuidad del sistema hacia nuevas funcionalidades, manteniendo una base estable y verificable.

El versionamiento también permite respaldar las decisiones técnicas tomadas durante el desarrollo, como la implementación del procesamiento concurrente, la elección del modelo de regresión logística, la generación de evidencias reproducibles y la organización modular del código.

11. CONCLUSIONES

1. El sistema desarrollado logró procesar el dataset original *Individual Household Electric Power Consumption*, compuesto por 2,075,259 registros. Después del proceso de limpieza y validación, se conservaron 2,049,280 registros válidos y se descartaron 25,979 registros, principalmente por presencia de valores faltantes.

2. La implementación demuestra concurrencia real y verificable en Go. La carga y limpieza de datos utilizan un patrón productor/consumidor con goroutines, channels y workers, mientras que el entrenamiento del modelo divide el cálculo de gradientes en tareas paralelas que posteriormente son reducidas por un coordinador.
3. La variable objetivo *high_demand* fue definida de forma objetiva mediante el percentil 75 de *Global_active_power*, equivalente a 1.528 kW. Esta decisión permitió transformar el problema en una tarea de clasificación binaria orientada a identificar momentos de alta demanda eléctrica doméstica.
4. El modelo de regresión logística binaria implementado en Go obtuvo un accuracy de 0.8559, una precisión de 0.8941, un recall de 0.4805 y un F1-score de 0.6251. Estos resultados evidencian que el clasificador es funcional y presenta alta confiabilidad cuando predice eventos de alta demanda.
5. La función de pérdida disminuyó desde 0.693147 hasta 0.358911 durante el entrenamiento, lo que demuestra que el proceso de optimización converge y que el modelo aprende patrones presentes en los datos procesados.
6. El análisis exploratorio permitió identificar patrones relevantes de consumo, especialmente picos de demanda entre las 19:00 y 21:00, mayor consumo promedio durante fines de semana y variaciones mensuales asociadas a estacionalidad. Estos hallazgos refuerzan la relación del proyecto con la sostenibilidad energética doméstica.
7. El proyecto mantiene una arquitectura modular implementada en Go, con separación clara entre carga, preprocesamiento, generación de variables, entrenamiento, métricas, análisis y visualización. Esta estructura facilita la reutilización del modelo, la reproducción experimental y la evolución futura hacia servicios de predicción, procesamiento distribuido, almacenamiento persistente y visualización web.

12. RECOMENDACIONES

1. Implementar una API REST en Go que cargue el archivo *modelo_entrenado.json* y exponga un endpoint de predicción de alta demanda eléctrica a partir de nuevas mediciones del usuario.
2. Extender la arquitectura hacia un cluster de nodos de Machine Learning, donde un coordinador distribuya tareas de procesamiento o predicción y consolide resultados parciales mediante comunicación interna.
3. Incorporar almacenamiento persistente para registrar predicciones, métricas históricas, consultas realizadas y resultados de análisis, facilitando auditoría y seguimiento del sistema.
4. Evaluar ajustes en el umbral de decisión del clasificador para mejorar el recall y detectar una mayor proporción de eventos reales de alta demanda, manteniendo un equilibrio adecuado con la precisión.
5. Analizar estrategias de mejora del modelo, como ponderación de clases, balanceo del dataset, incorporación de nuevas variables temporales o comparación con otros modelos interpretables.

13. BIBLIOGRAFÍA

Hebrail, G., & Berard, A. (2006). Individual Household Electric Power Consumption [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C58K54>

The Go Authors. (2026). The Go Programming Language Documentation. <https://go.dev/doc/>

Docker Inc. (2026). Docker documentation. <https://docs.docker.com/>

14. ANEXOS

Anexo A. Link del repositorio en GitHub

Repositorio remoto del proyecto:

<https://github.com/Omar362267/TB2-Programaci-n-Concurrente-.git>

Este repositorio contiene el código fuente del sistema, los módulos implementados en Go, los scripts de ejecución, la documentación técnica y la estructura del proyecto.

Anexo B. Archivos de resultados principales

La siguiente tabla resume los archivos generados por el sistema y su utilidad dentro del informe.

Archivo	Descripción
results/final_run/resumen_limpieza.json	Resumen de la limpieza concurrente, incluyendo registros leídos, válidos, descartados y motivos de descarte.
results/final_run/metricas_modelo.json	Métricas del modelo, matriz de confusión, loss inicial, loss final e historial de entrenamiento.
results/final_run/modelo_entrenado.json	Pesos, sesgo, parámetros de normalización y configuración del modelo entrenado.
results/final_run/predicciones_muestra.csv	Muestra de predicciones generadas por el clasificador.
data/processed/features_high_demand.csv	Dataset procesado con variables predictoras y variable objetivo high_demand.
results/analysis/estadisticas_descriptivas.csv	Estadísticas descriptivas de las principales variables eléctricas.
results/analysis/matriz_correlacion.csv	Matriz de correlación entre variables numéricas.
results/analysis/consumo_promedio_hora.csv	Agregación del consumo promedio y tasa de alta demanda por hora.

results/analysis/consumo_por_dia_semana.csv	Agregación del consumo promedio por día de semana.
results/analysis/consumo_por_mes.csv	Agregación del consumo promedio por mes.
results/figures/*.svg	Gráficos generados en Go para análisis exploratorio, evaluación y evidencias visuales.
results/benchmarks/*	Resultados de las pruebas comparativas con distinta cantidad de workers.

Anexo C. Extracto de salida de predicciones

La siguiente tabla presenta una muestra de predicciones generadas por el modelo. La columna actual_high_demand representa la clase real y la columna predicted_high_demand representa la predicción del clasificador.

index	actual_high_demand	predicted_high_demand
0	0	0
1	0	0
2	0	0
3	0	0
4	1	1
5	0	0
6	1	1
7	1	0
8	0	0
9	1	1
10	1	0
11	0	0
12	0	0
13	0	0
14	0	0
15	0	0
16	1	0
17	1	1
18	0	0
19	1	1

Este extracto permite observar casos correctamente clasificados y casos donde el modelo no detecta eventos reales de alta demanda. Esto es coherente con el recall obtenido y con la interpretación de que el modelo es conservador.

Anexo D. Extracto de historial de entrenamiento

El historial de entrenamiento muestra la evolución de la función de pérdida durante las iteraciones del modelo.

Iteración	Loss
0	0.693147
1	0.693147
2	0.611278
5	0.560037
10	0.533733
25	0.476996
50	0.417936
75	0.382467
100	0.358911

La disminución sostenida del loss evidencia que el entrenamiento converge y que el modelo aprende patrones a partir de los datos procesados.

Anexo E. Evidencias de benchmark concurrente

Para evaluar el rendimiento del sistema se ejecutaron pruebas variando la cantidad de workers. Estas pruebas permiten comparar el comportamiento del pipeline bajo diferentes niveles de concurrencia.

Configuración	Evidencia esperada	Ubicación
1 worker	Resultado base de ejecución secuencial o con mínima concurrencia	results/benchmarks
2 workers	Resultado con paralelismo moderado	results/benchmarks
4 workers	Resultado con mayor distribución de trabajo	results/benchmarks
8 workers	Configuración principal de ejecución concurrente	results/benchmarks

Gráfico comparativo	Comparación visual de tiempos o registros por segundo	results/figures/benchmark_wor kers.svg
---------------------	---	---

Anexo F. Estructura de carpetas esperada para revisión

La estructura del proyecto se organizó para separar código fuente, datos, resultados, análisis, visualizaciones, scripts y documentación.

Carpeta o módulo	Propósito
cmd/pc3	Ejecutable principal del pipeline.
cmd/pc3-analysis	Comando para análisis estadístico y generación de gráficos.
cmd/pc3-benchmark	Comando para pruebas comparativas de rendimiento.
internal/loader	Carga concurrente del dataset.
internal/preprocessing	Limpieza, parseo y validación de registros.
internal/features	Generación de variables predictoras y target.
internal/ml	Entrenamiento del modelo de regresión logística.
internal/metrics	Cálculo de métricas de evaluación.
internal/analysis	Estadísticas descriptivas y agregaciones del dataset.
internal/visualization	Generación de gráficos SVG desde Go.
data/raw	Dataset original.
data/processed	Dataset procesado.
results/final_run	Resultados de la ejecución final.
results/analysis	Tablas de análisis exploratorio.
results/figures	Gráficos generados por el sistema.
results/benchmarks	Resultados de pruebas de rendimiento.
scripts	Scripts de ejecución para PowerShell y Bash.
docs	Documentación técnica y guías de ejecución.
docker	Archivos de preparación para contenerización.

Anexo G. Evidencias visuales de ejecución y resultados

```

Ejecucion PC3 completada
Registros leidos: 2075259
Registros validos: 2049280
Registros descartados: 25979
Workers usados: 8
Muestras entrenamiento: 1639424
Muestras prueba: 409856
Loss inicial: 0.693147 | Loss final: 0.268712
Accuracy: 0.8887 | Precision: 0.8584 | Recall: 0.6645 | F1: 0.7491
Modelo guardado en: results\final_run\modelo_entrenado.json
Dataset procesado guardado en: data\processed\features_high_demand.csv
Resultados generados en: results/final_run

```

workers	rows_processed	load_ms	features_ms	train_ms	total_ms	rows_per_second	speedup	efficiency
1	2075259	3588	307	79874	83982	24710.64046	1	1
2	2075259	4582	364	42598	47711	43495.78614	1.760223	0.880112
4	2075259	2532	309	24103	27105	76563.1044	3.098395	0.774599
8	2075259	2355	320	15958	18855	110062.368	4.454097	0.556762